Guia de estudo da arquitetura IdeiaTech

Como o sistema foi construído e como suas partes trabalham juntas

Material didático independente do aplicativo. Explica a estrutura e o comportamento observados no código atual, com exemplos de como acompanhar uma requisição do navegador até o banco de dados e de volta.

EM UMA FRASE	STACK	PONTO IMPORTANTE
Aplicação web monolítica: uma aplicação Flask serve telas, recebe ações e conversa diretamente com SQLite.	Python 3 · Flask · Jinja · HTML/CSS · JavaScript sem framework · SQLite · Werkzeug	A triagem chamada IdeiaAI é determinística no código atual; não chama Groq/modelo de linguagem.

1. Um mapa mental do sistema

O navegador é a interface da pessoa. Ele recebe HTML e CSS, executa JavaScript e envia formulários ou chamadas HTTP ao servidor. O servidor Flask é a camada que decide o que a pessoa pode fazer, valida os dados, executa as regras de negócio, lê ou grava o SQLite e devolve uma página ou JSON. Não há uma API separada, microserviços, frontend React/Vue ou servidor de banco remoto nesta configuração.

NAVEGADOR	SERVIDOR FLASK	BANCO SQLITE
HTML/Jinja renderizado, CSS, imagens, Lucide e JavaScript. Estado temporário do carrinho no localStorage; passagem curta da análise para a solicitação via sessionStorage.	Rotas em BackEnd/app.py e BackEnd/orcamentos.py. Autenticação, autorização, validação, cálculo, transações e montagem da resposta.	Arquivo BackEnd/dados/ideia_tech.db. Contas, empresas, produtos, pedidos, solicitações, orçamentos, ordens e históricos.

A regra prática O navegador pode melhorar a experiência, mas o servidor é quem deve decidir se a ação é válida. Por exemplo, o preço mostrado no carrinho não é aceito como verdade quando o pedido é gravado: Flask consulta preço e estoque no SQLite.

2. Stack e papel de cada tecnologia

TECNOLOGIA	PAPEL NESTE PROJETO	ONDE OBSERVAR
Python 3	Linguagem do servidor e das regras de negócio.	BackEnd/*.py
Flask 3.x	Servidor web WSGI, roteamento HTTP, sessão, respostas, redirecionamentos e integração com templates.	BackEnd/app.py; requirements.txt
Jinja2	Templates HTML: substitui variáveis, percorre listas e exibe condições no servidor antes de entregar a página.	Pages/*.html com {{ }} e {% %}
HTML e CSS	Estrutura semântica das telas e estilos por página, com uma base comum e regras responsivas.	Pages/; Style/
JavaScript nativo	Interações no navegador: carrinho, formulários assíncronos, CSRF, CEP e atualização visual.	Script/*.js e scripts inline
SQLite	Banco relacional embutido em um arquivo, sem serviço de banco separado para o modo local.	BackEnd/schema.sql; BackEnd/banco.py
Werkzeug	Hash/verificação de senhas e utilitários que acompanham Flask.	generate_password_hash / check_password_hash em app.py
Lucide	Biblioteca local para transformar atributos de ícone em SVG.	Script/vendor/lucide.min.js; Script/icones.js
ViaCEP / Google Fonts	Dependências remotas usadas pelo navegador para consulta de CEP e fonte em algumas páginas.	Script/fluxoServico.js; páginas com fonts.googleapis.com
Groq	Pacote listado em requirements, sem chamada ativa encontrada no caminho do endpoint de análise.	requirements.txt; BackEnd/ia_prompt.py

Não há build de frontend O projeto não usa npm/Vite/Webpack para empacotar a interface. Os arquivos de Script, Style, Pages e Assets são servidos pelo Flask. O arquivo requirements.txt declara Flask e groq.

3. Pastas e responsabilidades

CAMINHO	RESPONSABILIDADE	COMO SE CONECTA
BackEnd/app.py	Ponto de composição do servidor: cria Flask, carrega configuração, conecta SQLite e declara a maior parte das rotas.	Importa/configura banco, registra rotas e inicia servidor local em 127.0.0.1:5000.
BackEnd/orcamentos.py	Rotas para listar e criar orçamento na área da empresa e permitir decisão do cliente.	Recebe app, conectar_banco e buscar_usuario_sessao; registra rotas no mesmo Flask.
BackEnd/schema.sql	Definição das tabelas, chaves, restrições e índices.	Executado por banco.py no bootstrap/migração.
BackEnd/banco.py	Cria diretório/banco, executa schema e migra estruturas antigas.	É importado por app.py e também pode ser executado manualmente.
BackEnd/configuracao.py	Lê BackEnd/config.local.json e preenche variáveis ausentes do ambiente.	app.py chama carregar_configuracao antes de ler a chave secreta.
BackEnd/frete.py	Regras simples de região por primeiro dígito do CEP e formatação de endereço.	Importado e chamado por rotas de frete e pedidos em app.py.
BackEnd/ia_prompt.py	Texto de persona/instruções que poderia ser usado com uma IA.	Nenhuma referência ativa encontrada a esse texto dentro do fluxo analisador atual.
BackEnd/tests/	Testes automatizados de integração do Flask e SQLite temporário.	test_pedidos.py e test_dados_empresa.py cobrem áreas específicas.
BackEnd/login.py, cadastrarUsuario.py, cadastrarEmpresa.py	Pequenos utilitários de terminal que interagem diretamente com SQLite.	Não são importados pelo app web; o uso normal é pelas rotas Flask.
Pages/	HTML público e templates Jinja dinâmicos, inclusive área da empresa.	render_template em rotas dinâmicas; send_from_directory para páginas/arquivos simples.
Script/	JavaScript de interação e biblioteca local de ícones.	HTML inclui os scripts por <script src>.
Style/	CSS global, por página e regras responsivas.	HTML inclui folhas base e específicas.
Assets/	Imagens, vídeos, ilustração, logo e recursos visuais.	URLs /Assets/... servidas pelo endpoint de arquivos permitidos.
BackEnd/dados/	Banco SQLite local e arquivos de backup.	Dados locais não são distribuídos como parte do deploy/repositório compartilhado.

Utilitários antigos cadastrarUsuario.py e login.py são scripts de terminal; não fazem parte das rotas web. cadastrarEmpresa.py grava a ficha empresarial diretamente e não estabelece sozinho o vínculo usuario_id. Para o percurso atual, prefira o cadastro empresarial pela rota Flask, que cria usuário e empresa vinculados.

4. Como uma requisição atravessa as camadas

1. A pessoa abre uma URL, clica em um botão ou envia um formulário.
2. O navegador faz GET, POST ou fetch para uma URL do mesmo servidor Flask.
3. Antes da função da rota, o gancho validar_csrf rejeita métodos de escrita sem um token de sessão válido. Outro gancho protege URLs /Pages/Empresa/ contra usuário anônimo ou conta cliente.
4. A função Flask busca a sessão, verifica o tipo de conta, valida entradas e abre uma conexão SQLite quando precisa dos dados.
5. As consultas usam parâmetros SQL; uma escrita altera registros e termina com commit. Em falha, alguns fluxos fazem rollback.
6. A rota escolhe a resposta: render_template para HTML dinâmico, jsonify para JSON, texto de erro ou redirect para outra tela.
7. O navegador atualiza a página ou interpreta JSON e atualiza elementos DOM com JavaScript.

Duas respostas comuns GET costuma carregar uma página/dados; POST tenta realizar uma mudança. Em POST de formulário tradicional, o padrão usado é validar, gravar e redirecionar (PRG: Post/Redirect/Get). Em operações assíncronas, como pedido e triagem, o JavaScript chama fetch e recebe JSON.

Templates dinâmicos versus arquivos estáticos

Flask usa Pages como pasta de templates. Uma rota explícita chama render_template quando precisa injetar dados ou executar Jinja. Existe também uma rota de arquivos que só permite as pastas Pages, Style, Script e Assets e usa send_from_directory. Isso

entrega o arquivo sem processar Jinja. Por isso, páginas que usam {{ variável }}, {% if %} ou url_for precisam ter uma rota explícita que as renderize; se fossem servidas diretamente pelo endpoint genérico, as chaves apareceriam como texto literal.

5. Inicialização, configuração e banco local

Ao importar BackEnd/app.py, carregar_configuracao lê BackEnd/config.local.json (quando existe) e só define variáveis que ainda não estejam no ambiente. Em desenvolvimento, o módulo pode criar esse arquivo local com chave secreta persistente, APP_ENV=development e recuperação local. O arquivo é ignorado pelo Git. Depois, o app exige IDEIATECH_SECRET_KEY e bloqueia RECUPERACAO_LOCAL junto com APP_ENV=production.

O import `import banco` executa código de topo de BackEnd/banco.py: cria BackEnd/dados se necessário, abre ideia_tech.db, liga foreign_keys, aplica schema.sql e verifica colunas para migrar bancos antigos. Algumas alterações também são descritas em COMO_RODAR.md como etapa manual. Empresas antigas sem usuario_id não são associadas automaticamente a uma conta, para evitar atribuição incorreta.

COMANDO LOCAL	O QUE FAZ
py -m venv .venv	Cria ambiente isolado do Python para dependências.
.venv\Scripts\python.exe -m pip install -r requirements.txt	Instala Flask e o pacote Groq declarados.
.venv\Scripts\python.exe BackEnd/banco.py	Executa a preparação/migração do banco local.
.venv\Scripts\python.exe BackEnd/app.py	Inicia o servidor de desenvolvimento em http://127.0.0.1:5000.
Ctrl+C no terminal	Encerra o servidor de desenvolvimento.

Persistência A configuração documentada é local, com SQLite por computador. Um backup existe como arquivos sob BackEnd/dados. Para operação real multiusuário, hospedagem e persistência compartilhada, deve-se planejar banco/backup/deploy adequados; esta configuração não prova isso.

6. Modelo de dados: o que as tabelas representam

TABELA	SIGNIFICADO E RELAÇÕES PRINCIPAIS
usuarios	Identidade e acesso. Tem nome, e-mail único, telefone, senha_hash, endereço, tipo cliente/empresa e versao_sessao. Empresas vinculam usuario_id.
empresas	Perfil empresarial ligado a um usuário por usuario_id (único). É a dona dos produtos, pedidos, solicitações e ordens.
produtos	Catálogo por empresa: nome, categoria, descrição, preço em centavos, estoque, imagem e ativo.
pedidos	Pedido por cliente e empresa, estado, total em centavos, modalidade/endereço/frete e carimbos de data.
itens_pedido	Linhas do pedido: produto, nome/preço capturados no momento da compra e quantidade.
solicitacoes	Pedido de atendimento do cliente a uma empresa: aparelho, problema, modalidade de chegada, dados de coleta e estados operacionais/orçamento.
orcamentos	Proposta associada de forma única a uma solicitação: diagnóstico, serviço, valor, prazo, taxa e resposta do cliente.
ordens_servico	Execução do reparo. Relaciona cliente, empresa e opcionalmente o orçamento aprovado; tem status e previsão.
historico_ordens	Eventos/status da ordem, observação e momento. Permite mostrar evolução ao cliente.
tokens_recuperacao_senha	Hash do token de recuperação, usuário, validade e momento de uso.
tentativas_login / tentativas_recuperacao	Registros de tentativas limitadas por identificador hash e horário.

Relacionamentos em linguagem simples

RELAÇÃO	LEITURA
usuario empresa → empresa	Uma conta empresarial fica vinculada a uma ficha de empresa; esse vínculo decide qual cadastro a conta administra.
empresa → produtos	Cada produto pertence a uma empresa. O catálogo público mostra produtos ativos.
cliente + empresa → pedido → itens	Uma compra é particionada por empresa; cada pedido guarda suas linhas, preço capturado, quantidade e frete.
cliente + empresa → solicitação → orçamento → ordem	A solicitação começa o atendimento. A empresa publica uma proposta. Aprovação do cliente cria a ordem e o primeiro registro do histórico.
ordem → histórico	Cada avanço de status pode adicionar uma observação cronológica da assistência.

Por que centavos? Preço e frete são armazenados como inteiros (por exemplo, 12.345 centavos), evitando imprecisão binária de números decimais em cálculos financeiros. Na tela, o valor é convertido e formatado como moeda.

7. Fluxo de autenticação e proteções

Cadastro cliente/empresa: formulários HTML enviam dados para /register/cliente ou /register/empresa. O servidor normaliza e valida campos, exige confirmação de senha/termos, calcula hash com Werkzeug e salva. A senha original não deve ser guardada. Cadastro empresarial cria tanto usuário tipo empresa como registro vinculado da empresa dentro do mesmo fluxo.

Login: POST /login normaliza e-mail, consulta usuário, verifica hash e usa hash falso para aproximar o custo quando o e-mail não existe. Identificadores de e-mail e IP são HMAC-SHA256 com a chave secreta e usados para limites de tentativa. No êxito, o servidor limpa a sessão anterior e grava usuario_id, versao_sessao e marca a sessão como permanente.

A sessão Flask padrão é um cookie assinado pelo SECRET_KEY: o conteúdo não é um registro de sessão SQLite. A aplicação ainda consulta o usuário a cada requisição protegida e compara versao_sessao. Se a versão mudou, limpa a sessão. Essa versão permite invalidar sessões existentes quando credenciais forem redefinidas.

CSRF: GET /csrf-token cria/retorna token aleatório guardado na sessão. O script protecaoCsrf.js busca o token antes de enviar formulários POST tradicionais; scripts AJAX incluem X-CSRF-Token. O gancho antes da rota compara o token recebido ao da sessão. Sessões usam HttpOnly e SameSite=Lax; o cookie Secure depende de APP_ENV=production.

Recuperação: o servidor sempre responde com texto genérico, limita tentativas por e-mail/IP, gera token aleatório e salva somente SHA-256 do token com expiração de 30 minutos. Em modo local, o link vai para o log/terminal. Em modo configurado para envio, usa SMTP/TLS. Ao trocar senha, marca tokens como usados, atualiza hash, incrementa versao_sessao e limpa a sessão atual.

Retorno de login Quando checkout exige login, o frontend guarda o destino temporariamente, encaminha para login e revalida o parâmetro `next` para manter retorno apenas a Pages do mesmo site. Isso evita redirecionamento aberto para domínio externo.

8. Fluxo de loja: catálogo, carrinho, frete e pedido

1. GET /Pages/vitrine.html consulta produtos ativos no SQLite e entrega HTML Jinja com nome, imagem, estoque e preço.
2. A página também carrega /Script/catalogoProdutos.js; apesar da extensão, esse endpoint gera JavaScript a partir dos produtos ativos do banco. Isso fornece ao carrinho um catálogo inicial para desenhar itens.
3. Script/carrinho.js guarda ids e quantidades em localStorage sob `ideiaTech.carrinho.v1`. O navegador pode editar visualmente quantidades; esse estado é apenas conveniência local.
4. Para entrega, o navegador consulta /api/frete?cep=... e salva/consulta endereço pelo /api/endereco. O módulo frete.py aplica tabela fixa pelo primeiro dígito do CEP e devolve valor em centavos/região.
5. Ao confirmar, fluxoAutenticacao.js pede /csrf-token e envia POST /pedidos com ids e quantidades. Se não houver login, pede autenticação e retorna ao carrinho.
6. O servidor confere papel cliente, formato e tamanho do carrinho, consulta produtos/estoque/preços no banco, confere ativo/disponibilidade e calcula total. Não confia no preço vindo do browser.
7. A transação começa com BEGIN IMMEDIATE, agrupa itens por empresa, cria um pedido por empresa, grava linhas com o preço daquele momento, reduz estoque condicionalmente e confirma tudo com commit. Em falha, faz rollback.
8. Resposta JSON com IDs dos pedidos e modalidade/frete; o frontend limpa carrinho e mostra confirmação. Cliente acompanha em MeusPedidos; a empresa vê seus pedidos e avança status permitido.

Limite do checkout O código registra pedido/estoque/status e oferece campo de modalidade de pagamento em outros fluxos. Não foi encontrado gateway de cartão/pagamento integrado; não trate confirmação do pedido como captura financeira.

9. Fluxo de assistência: triagem, orçamento e ordem de serviço

A. Triagem no navegador

Pages/Orçamento.html apresenta o formulário e o painel de análise. Script/analiseOrcamento.js valida marca/modelo/descrição no browser, envia JSON por fetch para POST /api/orcamento/analisar com token CSRF e mostra resposta. Os campos são escritos com textContent para evitar interpretar texto do usuário como HTML. A função atual em app.py verifica tamanho mínimo da descrição e devolve nome de serviço e referências conforme categoria. Ela não usa a variável de persona, pacote Groq nem um modelo de linguagem; por isso, é uma classificação/saída determinística, não inferência de IA.

Quando a análise termina, o JavaScript guarda aparelho, problema, serviço, categoria, modalidade e endereço em sessionStorage como `ideiatech_orcamento`. A página de atendimento lê esse rascunho para preencher campos. Esse dado ainda está no navegador; só vira registro persistente após enviar solicitação.

B. Solicitação persistida

Um cliente autenticado envia POST /Pages/solicitarAtendimento.html. O servidor lista empresas disponíveis, valida escolha, aparelho, relato, modalidade loja/coleta e endereço se necessário, grava em solicitacoes e redireciona para Acompanhar. O endereço de coleta é serializado como JSON textual no campo da tabela. ViaCEP é uma conveniência de navegador; os campos também podem ser digitados.

C. Proposta comercial

A empresa abre Solicitações e Orçamentos. A rota de /Pages/Empresa/orcamentos.html filtra apenas solicitações da empresa autenticada, evita criar proposta repetida e valida diagnóstico, serviço, prazo e valores com Decimal. O valor final é persistido em centavos. A página do cliente reúne os orçamentos da conta logada.

D. Decisão e início da ordem

Cliente aprova ou recusa em POST /Pages/MeusOrcamentos/<id>/decisao. A rota confirma que aquele orçamento pertence a uma solicitação do cliente atual e ainda aguarda aprovação. Em aprovação, cria uma ordem em ordens_servico e a primeira linha de historico_ordens na mesma transação. Em recusa, valida a modalidade de saída. O estado da solicitação é atualizado e o cliente volta ao acompanhamento.

E. Execução do serviço

A empresa avança o status da ordem e pode informar observação; as operações verificam se a ordem pertence à empresa logada e adicionam evento ao histórico. A página do cliente busca apenas suas próprias ordens, histórico, solicitações pendentes e orçamentos pendentes. Quando a ordem fica pronta, o cliente pode registrar preferência de retirada/devolução e pagamento informado (site/presencial); o campo registra preferência, não prova que um pagamento aconteceu.

ETAPA	REGISTRO MAIS RELEVANTE	QUEM ATUA
Relato enviado	solicitacoes (status inicial Nova solicitação)	Cliente
Proposta criada	orcamentos (Aguardando aprovação) e solicitação em análise	Empresa
Cliente decide	orcamentos status Aprovado/Recusado; solicitação concluída	Cliente
Reparo autorizado	ordens_servico + primeira linha historico_ordens	Sistema, na aprovação
Andamento/entrega	status da ordem + novas linhas de histórico	Empresa; cliente acompanha

10. Área empresarial e regra de propriedade dos dados

A área /Pages/Empresa/ tem um before_request de proteção global. Ele busca a conta no banco; sem sessão redireciona para login com next e uma conta cliente recebe 403. As rotas específicas voltam a consultar empresa pelo usuario_id e limitam queries pelo empresa_id. Assim, “estar logado” não basta: a conta deve ter o papel correto e a linha deve pertencer à empresa daquela conta.

TELA DA EMPRESA	RESPONSABILIDADE
painelEmpresa.html	Métricas derivadas do banco: serviços ativos, prontos para retirada, orçamentos pendentes e ordens recentes.
dadosEmpresa.html	Atualiza perfil da empresa e nome/telefone da conta vinculada.
aparelhosCadastrados.html	Lista informações de aparelhos/serviços relacionados.
solicitacoes.html	Lista solicitações recebidas pela empresa e contato/dados operacionais do cliente.
orcamentos.html	Cria proposta para solicitação ainda elegível e lista propostas existentes.
produtos.html	Cadastra/edita produto, seleciona imagem local, preço, estoque, ativa/desativa.
pedidos.html	Lista pedidos da empresa e permite mudanças de estado válidas.
ordensServico.html	Lista ordens e permite avançar status registrando histórico.

11. Frontend, templates, estilos e recursos

Pages contém HTML que combina markup e, nas rotas dinâmicas, sintaxe Jinja. O backend passa variáveis nomeadas para render_template, como usuario, produtos, ordens, orcamentos, empresa e csrf_token. Jinja escapa variáveis HTML por padrão. Links entre telas são majoritariamente caminhos absolutos /Pages/... ou relativos. Um arquivo parcial como _mensagens.html é incluído por outros templates para mensagens flash.

CSS usa Style/Style.css como base de variáveis e componentes comuns, folhas específicas para páginas e Style/responsivo.css para media queries em várias faixas de viewport e preferência de movimento reduzido. As classes e seletores do CSS dependem dos nomes e atributos data-* presentes no HTML. Os painéis empresariais possuem folha própria em Style/Empresa. O browser combina as folhas na ordem dos links; regra posterior e mais específica pode sobrescrever a anterior.

JavaScript é organizado em pequenos módulos IIFE (funções autoexecutadas), que só iniciam se encontram o elemento da página relevante. Isso permite carregar certos scripts em HTML sem acoplar todos os scripts a todas as telas. `defer` faz o navegador adiar execução até o HTML ser analisado. `icones.js` chama Lucide após carregar a biblioteca local.

SCRIPT	CONTRATO COM A PÁGINA / BACKEND
protecaoCsrf.js	Intercepta forms POST padrão, obtém token e o adiciona antes de submeter.
autenticacao.js	Trata formulários AJAX de cadastro/login: FormData + token, feedback acessível e redirecionamento.
fluxoAutenticacao.js	Protege checkout/login de retorno; confirma pedido com POST /pedidos.
carrinho.js	Lê catálogo, desenha carrinho, manipula localStorage, endereço/frete e botão de pedido.
analiseOrcamento.js	Valida dados da triagem, chama endpoint JSON, mostra resultado e grava rascunho em sessionStorage.
fluxoServico.js	Contador de texto, alternância de campos de coleta e consulta ViaCEP.
recuperarSenha.js	Alterna o formulário de pedido e o de nova senha conforme token na query string.
catalogoProdutos.js	Endpoint que gera um objeto global CatalogoProdutos baseado no banco; não é um catálogo hardcoded.
icones.js	Executa a biblioteca Lucide nos elementos com data-lucide.

12. Rotas principais para navegar pelo código

DOMÍNIO	ROTAS / PÁGINAS REPRESENTATIVAS
Entrada e segurança	GET /; GET /csrf-token; POST /login; POST /logout; POST /register/cliente; POST /register/empresa; POST /recuperar-senha; GET/POST /redefinir-senha.
Perfil/entrega	GET/POST /Pages/Perfil.html; GET/POST /Pages/editarPerfil.html; GET/POST /api/endereco; GET /api/frete.
Loja	GET /Pages/vitrine.html; GET /Script/catalogoProdutos.js; POST /pedidos; GET /Pages/MeusPedidos.html; GET /Pages/Empresa/pedidos.html e POST para status.
Assistência	POST /api/orcamento/analisar; GET/POST /Pages/solicitarAtendimento.html; GET /Pages/Acompanhar.html; GET /Pages/MeusOrcamentos.html e POST para decisão.
Operação da empresa	GET /Pages/Empresa/painelEmpresa.html; dadosEmpresa; aparelhosCadastrados; solicitacoes; orcamentos; produtos; ordensServico; status de pedido/ordem.
Arquivos de frontend	GET genérico limitado às pastas Pages, Style, Script e Assets. Arquivos enviados com send_from_directory.

Como achar uma rota Procure o decorator `@app.get`, `@app.post` ou `@app.route` em BackEnd/app.py. O nome da função imediatamente abaixo é o handler. Para orçamento, veja BackEnd/orcamentos.py e a chamada registrar_rotas_orcamentos no fim de app.py.

13. Decisões importantes e limitações da implementação

DECISÃO ATUAL	POR QUE É ÚTIL	O QUE NÃO SIGNIFICA
Monólito Flask + SQLite	Poucas partes para instalar e executar numa demo; simples de estudar e transportar localmente.	Não é arquitetura distribuída nem prova de escala para alta concorrência.
Status como texto com CHECK	Fácil mostrar fluxo de negócio e restringir valores aceitos no banco.	Nem todos os fluxos têm máquina de estados genérica; regras vivem nas rotas.
Preço em centavos	Cálculo financeiro determinístico e armazenamento sem fração decimal.	Não integra impostos, gateway de pagamento ou tabela logística externa.
Carrinho local no browser	Permanece no mesmo navegador e permite editar sem cadastro imediato.	Não sincroniza entre dispositivos nem é autoridade para estoque/preço.

DECISÃO ATUAL	POR QUE É ÚTIL	O QUE NÃO SIGNIFICA
Frete por primeira casa do CEP	Regra simples para protótipo, sem cotação remota.	Não é cotação postal precisa; tabela é aproximada/provisória.
IA preparada parcialmente	Há persona em ia_prompt.py e dependência Groq no requirements.	Não há integração observada no endpoint atual; as respostas são fixas por categoria.
Cobertura de teste seletiva	Existem casos de segurança/estoque/isolamento empresarial com dados temporários.	20 testes não cobrem todas as páginas e fluxos de forma ponta a ponta.

14. Como os testes automatizados ajudam a entender regras

BackEnd/tests/test_pedidos.py cria um banco temporário, importa a aplicação Flask e exercita endpoints com o test_client. Os casos cobrem carrinho/itens inválidos, produto inexistente/inativo/sem estoque, carrinho com várias empresas, gravação e redução de estoque, controle de acesso e CSRF, mudanças de status, cancelamento com devolução de estoque e rollback se uma operação falhar. Cada assert mostra uma regra de negócio que o código deve manter.

BackEnd/tests/test_dados_empresa.py constrói usuários e empresas de teste e verifica sessão, tipo de conta, isolamento de uma empresa em relação a outra, atualização dos dados da conta autenticada, validação de campos, CNPJ duplicado, CSRF e conta sem empresa vinculada. Esses testes são exemplos de testes de integração: executam Flask e SQL de verdade contra banco temporário, sem abrir navegador real.

Limite da suíte Testes via test_client validam respostas HTTP e estado do banco em cenários programados. Eles não verificam layout renderizado, CSS em mobile, interação real do mouse/teclado, carregamento da fonte remota ou conectividade SMTP.

15. Como estudar o repositório por ordem

1. Leia COMO_RODAR.md e requirements.txt para entender execução e dependências.
2. Leia BackEnd/schema.sql e faça um desenho das relações usuario → empresa → produtos/pedidos e cliente → solicitação → orçamento → ordem.
3. Leia o bootstrap de BackEnd/banco.py e a configuração em configuracao.py.
4. Comece BackEnd/app.py por carregar configuração, criação do Flask, conectar_banco, CSRF, autenticação e guardas de sessão.
5. Siga o catálogo: rota vitrine → template Pages/vitrine.html → endpoint catalogoProdutos.js → Script/carrinho.js → POST /pedidos.
6. Siga a assistência: Pages/Orçamento.html → analiseOrcamento.js → endpoint em app.py → sessionStorage → solicitarAtendimento → rota de gravação.
7. Continue por BackEnd/orcamentos.py para proposta, resposta do cliente e conversão em ordem de serviço.
8. Leia as páginas da empresa e os CSS específicos para associar dados Jinja, IDs/classes e aparência.
9. Por fim, rode os testes e associe cada assert aos invariantes de negócio que ele protege.

Experimento didático recomendado

Escolha uma ação, por exemplo “adicionar produto e confirmar pedido”. Localize o botão no HTML, veja o listener em carrinho.js, confira a URL de fetch em fluxoAutenticacao.js, encontre o decorator da rota em app.py, siga a consulta e a transação no SQLite, identifique a resposta JSON e volte ao código que atualiza a tela. Esse método ensina a conexão entre camadas melhor do que ler um arquivo inteiro fora de contexto.

16. Vocabulário rápido

TERMO	SIGNIFICADO NESTE CONTEXTO
Rota / endpoint	URL + método HTTP que o Flask associa a uma função.
Handler	Função Python que processa uma rota.
Template	HTML Jinja preenchido no servidor com dados antes da resposta.
JSON	Formato de dados usado nas chamadas fetch/API.
Sessão	Dados de autenticação mantidos entre requisições por cookie assinado e verificados pelo servidor.
CSRF	Ataque que tenta induzir o navegador autenticado a enviar ação sem consentimento; token vinculando formulário à sessão reduz o risco.

TERMO	SIGNIFICADO NESTE CONTEXTO
Hash de senha	Representação unidirecional e lenta usada para verificar senha sem guardar o texto original.
Transação	Grupo de gravações que confirma por inteiro (commit) ou é desfeito (rollback).
Foreign key	Relação de integridade referencial entre registros de tabelas.
localStorage / sessionStorage	Armazenamento do navegador: o primeiro persiste até limpeza; o segundo dura a sessão da aba. Não substituem o banco.

17. Para lembrar

- Flask recebe requisições e aplica as regras; o navegador cuida da interação e apresentação; SQLite guarda os registros persistentes.
- O frontend pode validar cedo e melhorar a experiência, mas valores, identidade, estoque e propriedade precisam ser verificados no backend.
- Cliente e empresa compartilham o sistema, mas cada rota verifica papel e propriedade dos registros.
- A cadeia principal de assistência é solicitação → orçamento da empresa → decisão do cliente → ordem de serviço → histórico.
- A cadeia principal da loja é catálogo ativo → carrinho local → validação no servidor → pedido por empresa → reserva de estoque.
- O código tem estrutura de prompt e pacote Groq, mas a triagem atual não usa modelo generativo.
- A aplicação foi documentada e configurada para execução local de desenvolvimento; produção exige infraestrutura e configuração específicas.

18. Perguntas para conferir seu entendimento

PERGUNTA	PISTA PARA MONTAR A RESPOSTA
Por que o backend consulta o produto outra vez ao receber um pedido?	O estado do carrinho é local e pode ser alterado; preço e estoque oficiais estão no SQLite.
Quais verificações acontecem antes de um POST protegido?	O token CSRF é comparado com a sessão; depois a rota verifica login, tipo de conta e propriedade do registro.
O que muda quando o cliente aprova um orçamento?	A proposta muda para aprovada, a solicitação é atualizada, e uma ordem e seu evento inicial são criados numa transação.
O que se perde se sessionStorage for limpo antes de enviar atendimento?	O rascunho da análise pode sumir do browser; ainda não havia sido persistido em solicitacoes.
Como impedir que uma empresa veja os pedidos de outra?	Resolver a empresa pelo usuario_id da sessão e filtrar cada consulta pelo id dessa empresa.
O que diferencia a tela da API de análise de uma IA conectada?	A tela faz fetch; a rota atual mapeia categoria para saídas fixas e não chama provedor de modelo.
O que significa rollback num pedido com vários produtos?	Desfaz pedido/itens/reduções já feitos na transação para evitar pedido parcial ou estoque incoerente.